⚙ Informe 03: Guía Técnica de Arquitectura, Requisitos y Manual de Despliegue

Especificación de Microservicios, Instalación y Puesta en Marcha

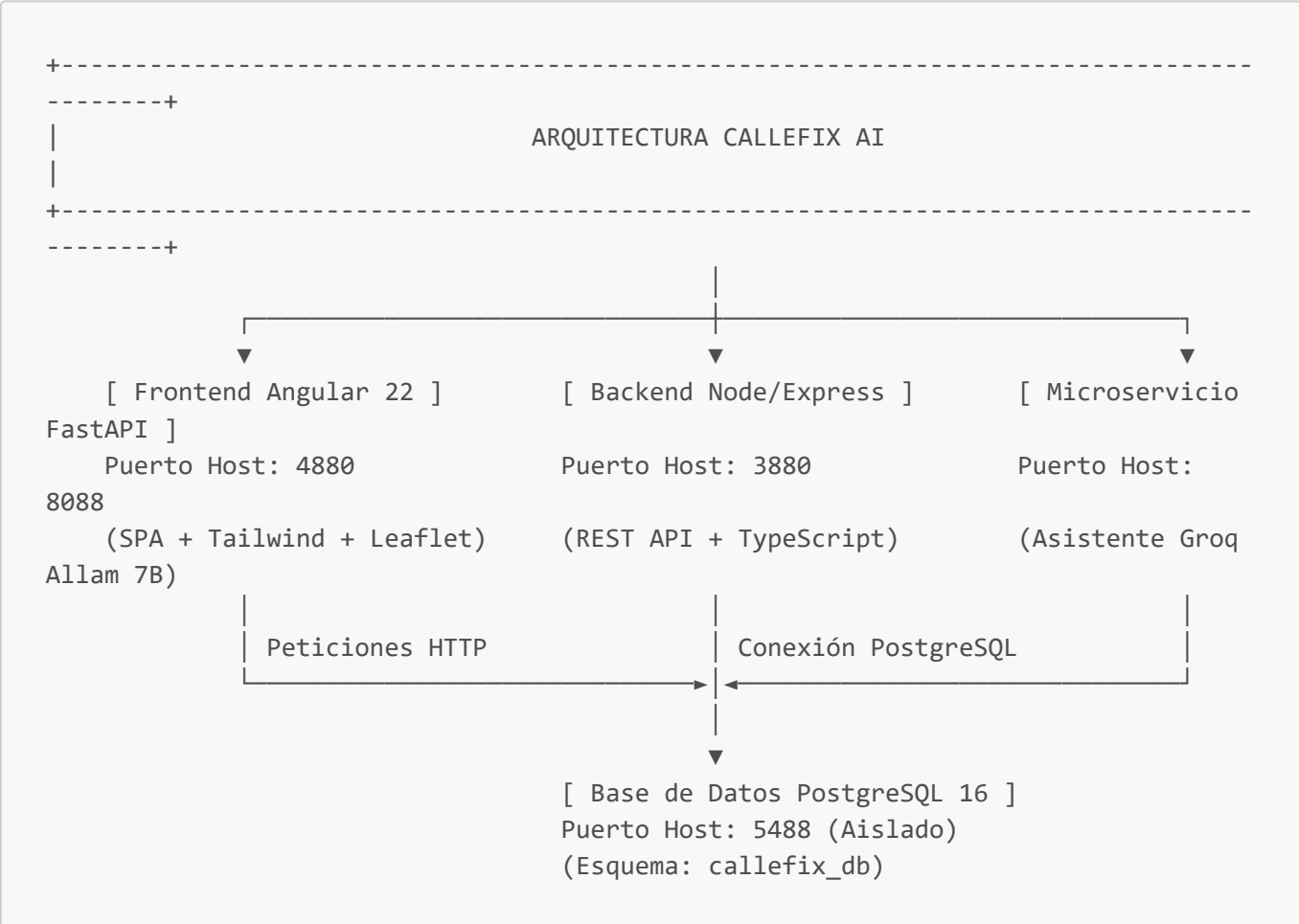
Plataforma: ¡Tu voz ha llegado! — CalleFix AI Málaga

> **Ámbito:** Despliegue de Sistemas, Contenerización Docker y Desarrollo Local
> **Fecha:** Convocatoria Hackathon Líderes Digitales 2026

---

1. Arquitectura Técnica y Aislamiento de Puertos (Port Isolation)

Para asegurar que la plataforma pueda ejecutarse de forma concurrente con otros proyectos o bases de datos sin generar colisiones de red, se implementó una política de **puertos exclusivos y desacoplados**:



Matriz de Puertos y Contenedores Docker:

Servicio	Imagen Base	Puerto Anfitrión (Host)	Puerto Interno	Variable Clave

Servicio	Imagen Base	Puerto Anfitrión (Host)	Puerto Interno	Variable Clave
Frontend Web	`node:22-alpine` / `nginx:alpine`	**`4880`**	`80`	`http://localhost:4880`
Backend API	`node:22-alpine`	**`3880`**	`3880`	`PORT=3880`, `DATABASE_URL`
Base de Datos	`postgres:16-alpine`	**`5488`**	`5432`	`POSTGRES_DB=callefix_db`
Microservicio IA	`python:3.11-slim`	**`8088`**	`8000`	`GROQ_MODEL=allam-2-7b`

- **Red Aislada de Docker:** `callefix-isolated-network` (driver bridge).
- **Volúmenes Persistentes:** `callefix-pgdata` (datos SQL) y `callefix-uploads` (imágenes de incidencias).

---

2. Requisitos y Herramientas Necesarias (Prerequisites)

Para ejecutar el proyecto de forma local o en servidor, el entorno debe contar con:

1. **Node.js y Gestor de Paquetes:**
   - Versión requerida: `Node.js >= 20.10.0 LTS` (Recomendado v22.x).
   - Gestor: `npm >= 10.x` o `pnpm >= 9.x`.
   - Comprobación:

```
node -v
npm -v
```

2. **Python (para microservicio de IA local):**
   - Versión requerida: `Python >= 3.10` (Recomendado 3.11 o 3.12).
   - Comprobación:

```
python --version
pip --version
```

3. **Docker y Docker Compose (para despliegue automatizado):**
   - `Docker Engine >= 24.0.0`
   - `Docker Compose v2 >= 2.20.0`

---

3. Guía de Instalación Paso a Paso (`npm install`)

Paso 1: Clonar el repositorio y acceder a la raíz

```
git clone git@github.com:Eng-Ahmet/lider-de-tecnologia.git
cd lider-de-tecnologia
```

Paso 2: Instalación de dependencias del Backend (Node.js)

```
cd backend
npm install
```

Librerías principales instaladas:

- `express` (^4.21.2): Servidor web y enrutamiento REST.
- `pg` (^8.13.3): Cliente relacional de PostgreSQL con soporte de transacciones y pools.
- `bcryptjs` (^3.0.3): Cifrado unidireccional de contraseñas de usuarios.
- `jsonwebtoken` (^9.0.3): Firma y validación de tokens JWT de sesión.
- `multer` (^1.4.5-lts.1): Procesamiento seguro de imágenes y archivos multipart.
- `tsx` (^4.19.3): Ejecutor TypeScript nativo sin transpilación previa en caliente.

Paso 3: Instalación de dependencias del Frontend (Angular 22)

```
cd ../frontend
npm install
```

Librerías principales instaladas:

- `@angular/core`, `@angular/router`, `@angular/forms` (^22.2.0): Framework reactivo.
- `leaflet` (^1.9.4): Motor de mapas interactivos sin costes de licencia.
- `tailwindcss` (^3.4.17): Motor de estilos utilitarios modernos.
- `rxjs` (~7.8.0): Programación reactiva para llamadas HTTP asíncronas.

Paso 4: Instalación de dependencias del Asistente IA (FastAPI)

```
cd ../fastapi-ai
pip install -r requirements.txt
```

Librerías principales instaladas:

- `fastapi` (>=0.110.0), `uvicorn` (>=0.28.0), `pydantic` (>=2.0.0), `groq` (>=0.9.0).

---

4. Modos de Ejecución del Sistema

Modo A: Despliegue Automatizado con Docker Compose (Recomendado)

Es la forma más rápida y limpia de evaluar el proyecto completo, ya que inicializa la base de datos, compila el frontend en Nginx y levanta todos los microservicios en puertos aislados:

```
# Desde el directorio raíz del proyecto:
docker compose up --build
```

- **Frontend Ciudadano:** http://localhost:4880
- **Backend API REST:** http://localhost:3880/api/health
- **Microservicio IA FastAPI:** http://localhost:8088/health
- **Base de Datos PostgreSQL:** Puerto host `5488`

Modo B: Ejecución en Entorno de Desarrollo Local

1. Iniciar Base de Datos PostgreSQL:

```
docker run -d --name callefix-db -p 5488:5432 -e POSTGRES_USER=callefix -e
POSTGRES_PASSWORD=callefix123 -e POSTGRES_DB=callefix_db postgres:16-alpine
```

2. Poblar la Base de Datos con Datos Reales de Málaga (Seeding):

```
cd backend
npm run seed
```

3. Iniciar el Servidor Backend (Puerto 3880):

```
cd backend
npm run dev
```

4. Iniciar la Aplicación Frontend Angular (Puerto 4880):

```
cd frontend
npm start -- --port 4880
```

5. Iniciar el Asistente IA FastAPI (Puerto 8000):

```
cd fastapi-ai
python -m uvicorn main:app --host 0.0.0.0 --port 8000
```

5. Credenciales de Acceso para Pruebas (Test Credentials)

Perfil	Correo Electrónico	Contraseña	Permisos y Capacidades
Administrador Municipal	`admin@callefix.malaga.es`	`Admin123!`	Visualización de auditoría completa, modificación de estados (`UnderReview` ➔ `InProgress` ➔ `Resolved`), notas internas de cuadrilla.
Ciudadano / Vecino	`vecino@malaga.es`	`Vecino123!`	Notificación de desperfectos con GPS en tiempo real, consulta de "Mis Reportes", chat con el asistente.

6. Catálogo de Endpoints de la API REST

Método	Ruta	Autorización	Propósito
`GET`	`/api/health`	Público	Comprobación de estado del servidor y conexión a PostgreSQL.
`POST`	`/api/auth/login`	Público	Autenticación y generación de JWT con rol de usuario.
`GET`	`/api/reports`	Bearer JWT	Listado paginado y filtrable de incidencias viales.
`POST`	`/api/reports`	Bearer JWT	Creación de reporte con subida de fotografía y geolocalización.
`GET`	`/api/reports/:id`	Bearer JWT	Detalle completo de incidencia, vecinos agrupados e historial de auditoría.
`PATCH`	`/api/reports/:id/status`	Admin JWT	Transición de estado municipal y registro de notas de cuadrilla.
`GET`	`/api/dashboard/stats`	Bearer JWT	Cálculo de métricas agregadas y porcentaje de ahorro por duplicados.
`POST`	`/api/ai/chat`	Público	Consulta conversacional al asistente municipal (respuestas ≤ 35 palabras).